Introduction to Data Structures

COMMON DATA STRUCTURE IN PYTHON

What are Data Structures?

- Data structures are ways of organizing and storing data so that it can be accessed and worked with efficiently.
- They are essential for managing large amounts of data and allow for effective data retrieval and manipulation.

Why are Data Structures Important?

- **Efficient Data Management**: They allow for faster data processing and retrieval.
- **Optimized Performance**: Well-chosen data structures reduce the time complexity of algorithms.
- **Data Organization**: Help maintain order, especially when handling large datasets.
- **Memory Utilization**: Good data structures minimize memory wastage.

Python's Built-in Data Structures

- **Lists**: Ordered, mutable collection.
- **Tuples**: Ordered, immutable collection.
- **Sets**: Unordered, mutable collection of unique elements.
- **Dictionaries**: Unordered, mutable collection of key-value pairs.

Introduction to Lists in Python

UNDERSTANDING PYTHON'S MOST VERSATILE DATA TYPE

What is a List in Python?

- A list is a collection of items in a particular order.
- Lists can hold items of different data types.
- Syntax: my_list = [item1, item2, item3, ...]

- Lists are ordered, changeable (Mutable), and allow duplicate elements.
- Lists are defined using square brackets[ ]

Creating a List

- Lists can contain items of any data type.

```
fruits = ["apple", "banana", "cherry"]
numbers = [1, 2, 3, 4, 5]
mixed = ["apple", 1, True, 3.14]
```

Accessing List Items

- Access elements using their index.

```
fruits = ["apple", "banana", "cherry"]
print(fruits[0])  # Output: apple
print(fruits[-1]) # Output: cherry
```

- Indexing starts at 0.
- Negative indexing allows you to access elements from the end.

Adding Elements to a List

APPEND() METHOD

- Adds an element to the end of the list.
- Syntax:
  list_name.append(item)

```
fruits = ["apple", "banana", "cherry"]
fruits.append("orange")
print(fruits)  # Output: ["apple", "banana", "cherry", "orange"]
```

- The append() method is used when you want to add a new item to the end of an existing list.
- The list is modified in place, meaning the original list is updated.

Adding Elements to a List

INSERT() METHOD

- Inserts an element at a specified index in the list.
- Syntax: list_name.insert(index, item)

```
fruits = ["apple", "banana", "cherry"]
fruits.insert(1, "grape")
print(fruits)  # Output: ["apple", "grape", "banana", "cherry"]
```

- The insert() method allows you to place an element at any position in the list.
- The index is where the new item will be added, and the items after the specified index are shifted one position to the right.

Removing Items from a List

Removing by Value:

- remove(): Removes the first occurrence of a specified value from the list.
- The remove() method finds the first instance of the specified value and removes it.
- If the value is not found, a ValueError is raised.

```
fruits = ["apple", "banana", "cherry", "banana"]
fruits.remove("banana")
print(fruits)  # Output: ["apple", "cherry", "banana"]
```

Removing Items from a List

Removing by Index :

- Removes and returns the element at a specified index (or the last element if no index is provided).

```
fruits = ["apple", "banana", "cherry"]
popped_item = fruits.pop(1)
print(fruits)  # Output: ["apple", "cherry"]
print(popped_item)  # Output: banana
```

Removing Items from a List

del Keyword :

- Deletes an element at a specified index or deletes the entire list.
- Unlike pop(), del does not return the deleted element.

```
fruits = ["apple", "banana", "cherry"]
del fruits[1]
print(fruits)  # Output: ["apple", "cherry"]
```

```
del fruits
# Now the list no longer exists, and trying to print it will raise an error
# print(fruits)  # This would raise a NameError since fruits is deleted
```

del V/S remove() the V/S pop()

del	remove()	pop()
del is a keyword.	It is a method.	pop() is a method.
To delete value it uses the index.	To delete value this method uses the value as a parameter.	This method also uses the index as a parameter to delete.
The del keyword doesn't return any value.	The remove() method doesn't return any value.	pop() returns deleted value.
The del keyword can delete the single value from a list or delete the whole list at a time.	At a time it deletes only one value from the list.	At a time it deletes only one value from the list.
It throws index error in case of the index doesn't exist in the list.	It throws value error in case of value doesn't exist in the list.	It throws index error in case of an index doesn't exist in the list.

List Slicing

- **Slicing Syntax**: my_list[start:end:step]

```
numbers = [1, 2, 3, 4, 5, 6]
print(numbers[1:4])    # Output: [2, 3, 4]
print(numbers[:3])     # Output: [1, 2, 3]
print(numbers[::2])    # Output: [1, 3, 5]
```

sort() Method

- Sorts the list in ascending order by default. It can also sort in descending order.
- Syntax: list_name.sort(reverse=False)
- The sort() method modifies the list in place.
- Use reverse=True to sort the list in descending order.

```
numbers = [5, 3, 8, 1, 2]
numbers.sort()
print(numbers)  # Output: [1, 2, 3, 5, 8]

# Sorting in descending order
numbers.sort(reverse=True)
print(numbers)  # Output: [8, 5, 3, 2, 1]
```

reverse() Method

- Reverses the elements of the list in place.
- Syntax: list_name.reverse()
- The reverse() method changes the order of the elements in the list to the reverse of their current order.
- It does not sort the list but simply reverses the order.

```
numbers = [1, 2, 3, 4, 5]
numbers.reverse()
print(numbers)  # Output: [5, 4, 3, 2, 1]
```

index() Method

- Returns the index of the first occurrence of a specified value.
- Syntax: list_name.index(item)
- The index() method returns the index of the first occurrence of the specified value.
- If the value is not found, it raises a ValueError.

```
fruits = ["apple", "banana", "cherry"]
index_of_cherry = fruits.index("cherry")
print(index_of_cherry)  # Output: 2
```

count() Method

- Returns the number of times a specified value occurs in the list.
- Syntax: list_name.count(item)
- The count() method is useful for counting how many times a specific value appears in a list.

```
numbers = [1, 2, 3, 4, 2, 2, 5]
count_of_twos = numbers.count(2)
print(count_of_twos)  # Output: 3
```

clear() Method

- Removes all elements from the list, resulting in an empty list.
- Syntax: list_name.clear()
- The clear() method empties the list, removing all elements.
- The list itself remains, but it becomes an empty list.

```
fruits = ["apple", "banana", "cherry"]
fruits.clear()
print(fruits)  # Output: []
```

Nested Lists

- A list can contain other lists as its elements.
- Nested lists are useful for representing complex structures like matrices.

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

print(matrix[0][1])  # Output: 2
```

THANK YOU

HAPPY LEARNING!